Bell Choir Lab – Nevin Fullerton

Project Description:

This project simulates a bell choir using Java multi threading. Each choir member holds one note and waits for the conductor to notify them to play their part for a specified length. The conductor loads a text file that holds all the notes and their corresponding lengths to be played, specified as a command-line argument. Then the conductor will notify the player who must play the note, wait for the player to finish, then go to the next player and notify them until all notes have been played.

Project Requirements:

This project meets the requirements determined by the lab assignment document and grading criteria.

Multiple Players:

For each unique note, there is a corresponding player who is responsible for playing that note. Each player waits and plays the note in its own thread.

Conductor:

The conductor determines which player is next, notifies them, and then waits for that player to finish their note. Once the player is finished, the player notifies the conductor and goes back to waiting. Then, the conductor goes to the next player until the song is done.

Loading External Note Sheet:

The program reads from a note sheet, either the default or a given one through arguments, and then plays the song. There is verification to ensure that the note sheet is in the proper format, notifying the user of any errors in the file and refusing to play it.

Naming and Formatting

The program uses proper Java naming conventions and is properly formatted for easier reading.

How To Run:

Prerequisites

Java 21
Apache Ant

Running

1. Pull down the GitHub repository into the local repository.
2. Type ‘ant run’ into the console. This will compile and run the program. Using the default song.
3. If you wish to play a different song, type ‘ant run -Dsong=“” ’ into the console, with the text file’s path in between the quotes.

Challenges Faced:

Challenge 1 – Player Access:

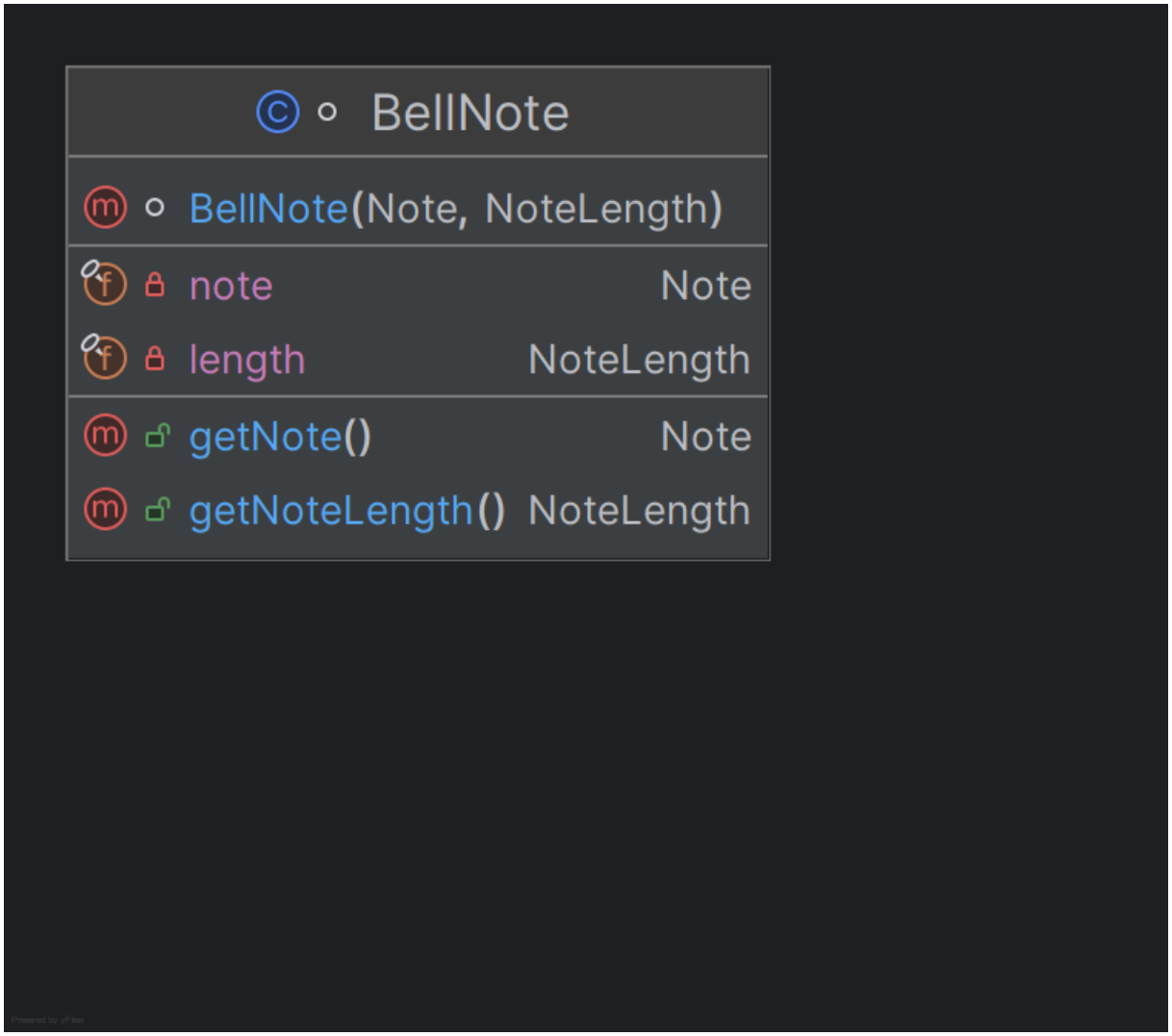
The first challenge was determining how players would be stored and how the conductor would be able to notify only one player at a time. My solution was to put all the players in a HashMap, having the key be the player’s note and the value be the player. This made accessing the player easy and quick. Then, afterwards, I had to figure out how to play notes.

Challenge 2 – Playing Notes:

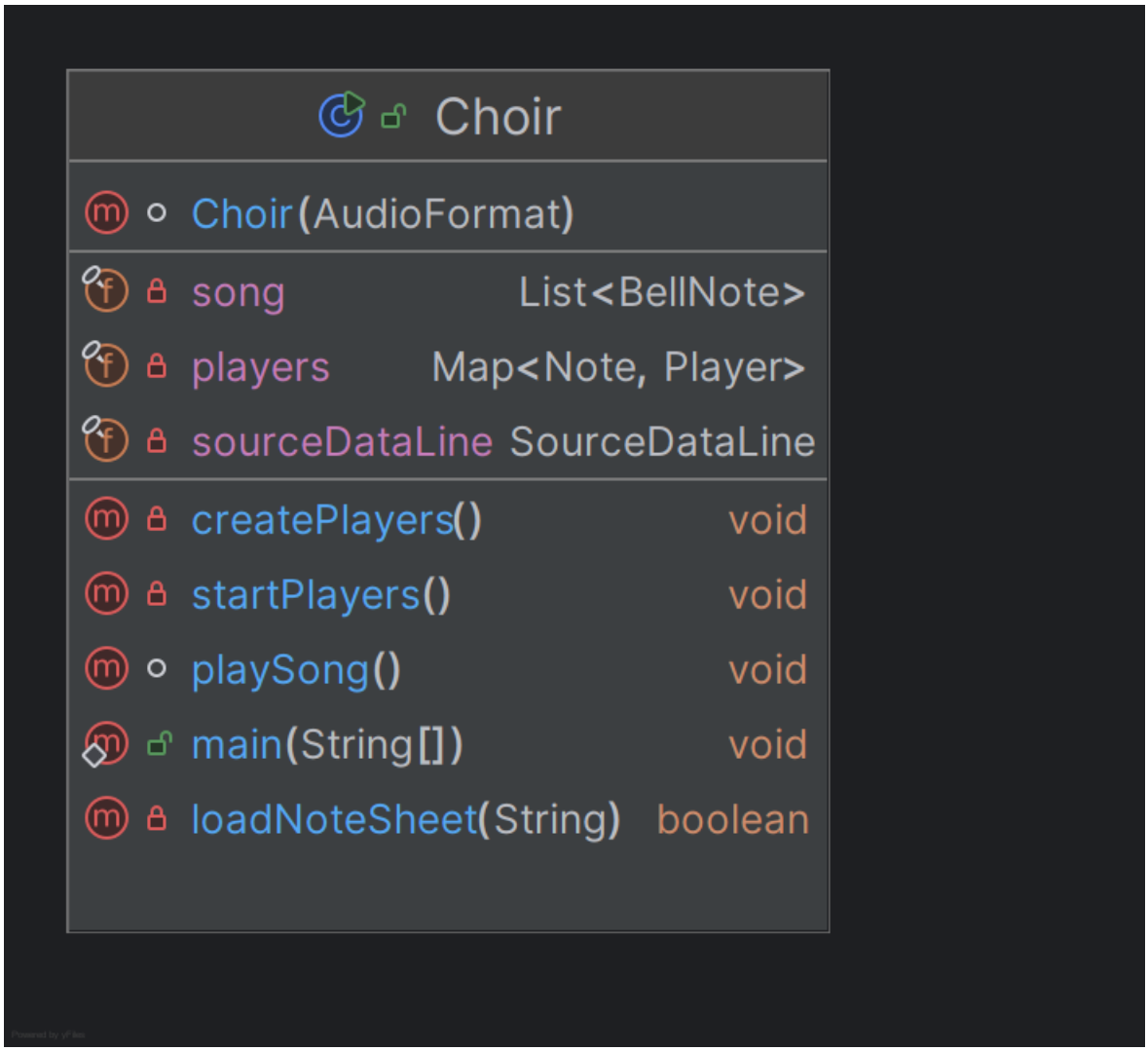
The second obstacle was how the players were to play their notes, and only have one play at a time. I solved this by passing a reference of the source data line to all players, so that they all could use the same one, with the conductor managing the starting and closing of the line. In order for only one to play, I made it so that all threads would run and then wait in their own threads. When the conductor wanted to play a song, he went to the player in a synchronized method, set their turn to true, notify the player, then wait for the player to finish playing their note. Once the note stopped playing, the player would set their turn to false and awaken the conductor. The conductor repeats this until the song is finished.

UML

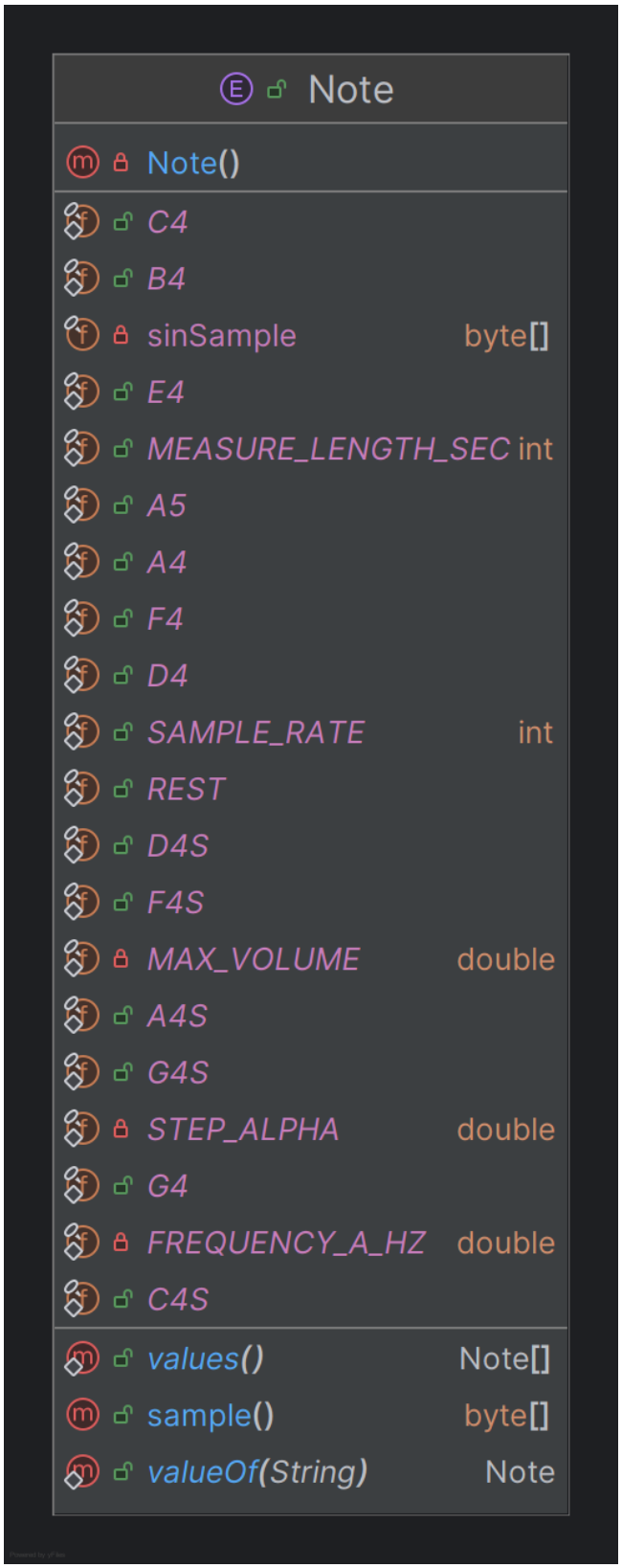
BellNote



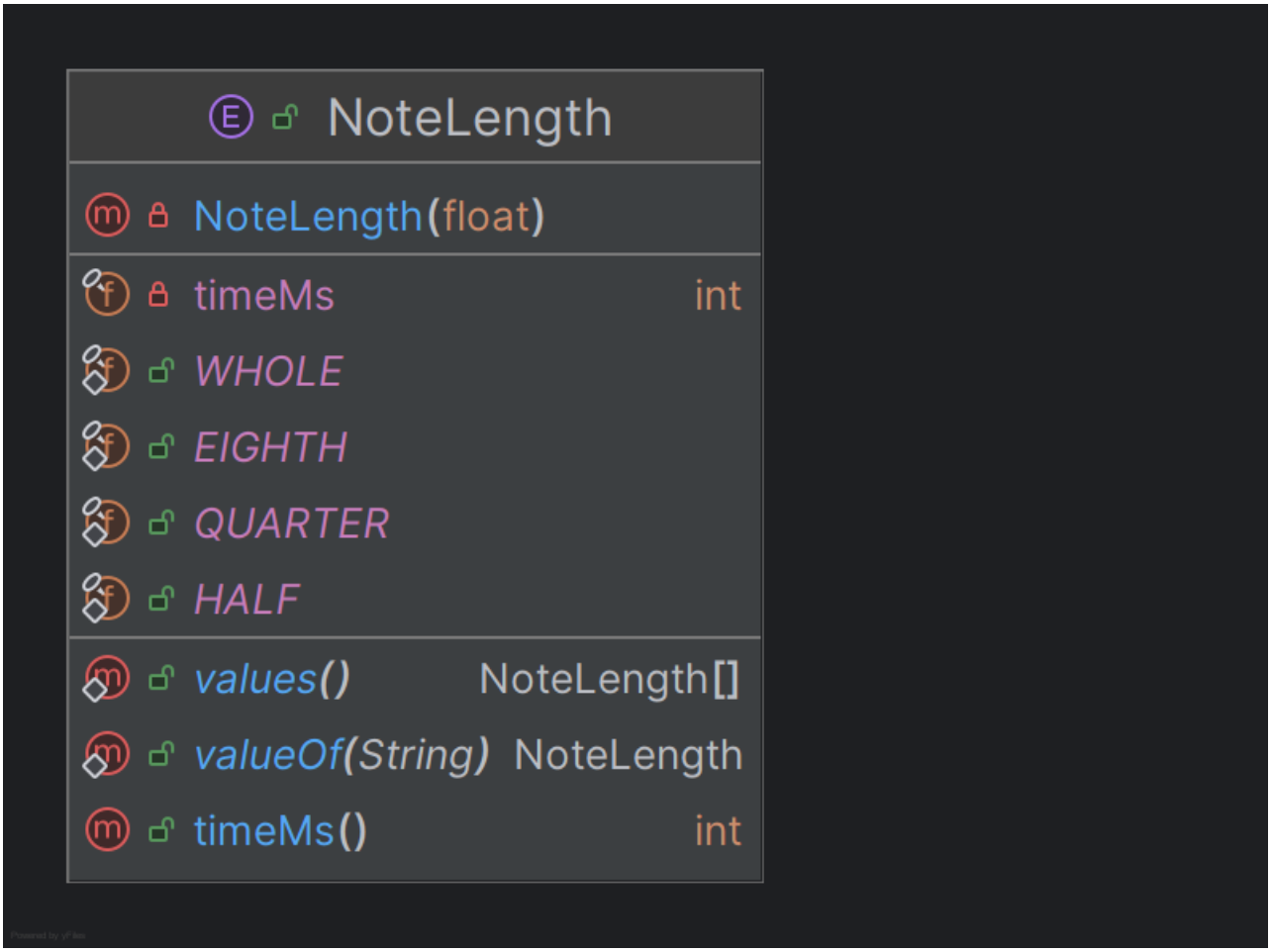
Choir


Choir
Choir(AudioFormat)
song List<BellNote>
players Map<Note, Player>
sourceDataLine SourceDataLine
createPlayers() void
startPlayers() void
playSong() void
main(String[]) void
loadNoteSheet(String) boolean

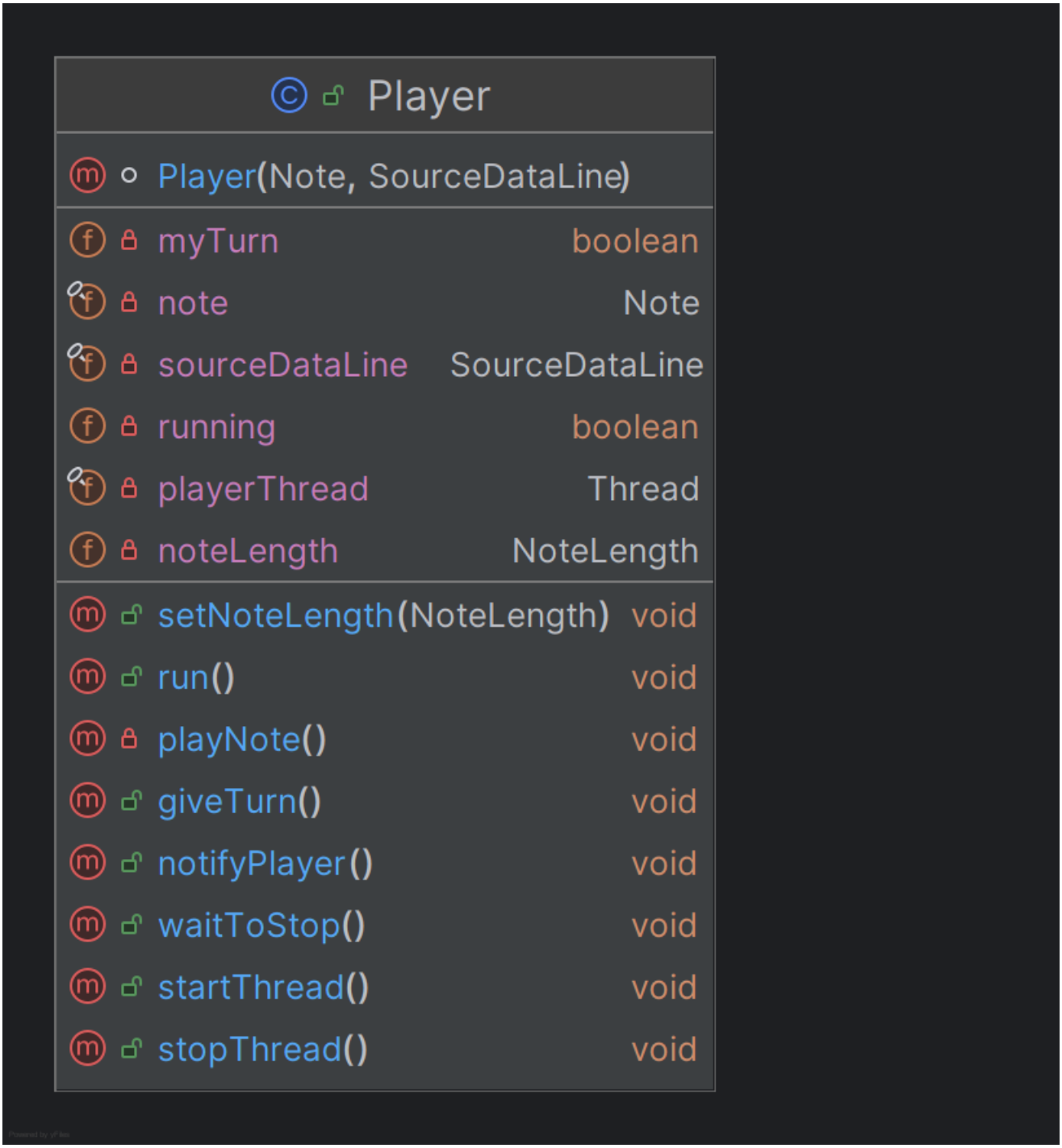
Note


Note
Note()
C4
B4
sinSample byte[]
E4
MEASURE_LENGTH_SEC int
A5
A4
F4
D4
SAMPLE_RATE int
REST
D4S
F4S
MAX_VOLUME double
A4S
G4S
STEP_ALPHA double
G4
FREQUENCY_A_HZ double
C4S
values() Note[]
sample() byte[]
valueOf(String) Note

NoteLength


NoteLength
NoteLength(float)
timeMs int
WHOLE
EIGHTH
QUARTER
HALF
values() NoteLength[]
valueOf(String) NoteLength
timeMs() int

Player


Player
Player(Note, SourceDataLine)
myTurn boolean
note Note
sourceDataLine SourceDataLine
running boolean
playerThread Thread
noteLength NoteLength
setNoteLength(NoteLength) void
run() void
playNote() void
giveTurn() void
notifyPlayer() void
waitToStop() void
startThread() void
stopThread() void